Last updated on ***Apr 27, 2026***

Unity Catalog business semantics

Unity Catalog business semantics is a centralized platform for defining and managing business metrics and KPIs. Standardized metric definitions promote consistent reporting across your organization and give AI tools the context they need to interpret your data accurately, with governance enforced through Unity Catalog.

Business semantics consists of two integrated components:

- **Metric views**: Reusable SQL objects that define and govern business KPIs.
- **Agent metadata**: Synonyms, display names, and formatting rules that help AI tools interpret your data in business terms.

Explore the tools and resources on this page to start working with Unity Catalog business semantics.

Get started

Use the following pages to create, query, and manage business semantics.

Unity Catalog metric views

Learn what metric views are, why they're more flexible than standard views, and how they fit into the Databricks platform.

Create and edit metric views

Define metric views using SQL DDL or the Catalog Explorer UI, with built-in YAML validation.

Query metric views

Query metric views from SQL editors, notebooks, dashboards, Genie Spaces, and external tools.

Tutorial: Build a complete metric view with joins

Walk through a complete sales analytics metric view with joins, measures, and agent metadata using the TPC-H dataset.

Model and optimize

Use the following pages to define and optimize your metric view data models.

Model metric views

Define sources, dimensions, measures, filters, and star and snowflake schema joins.

Advanced techniques for metric views

Build complex metrics using composability and window measures for time-series analysis.

Materialization for metric views

Pre-compute and incrementally refresh aggregations to improve query performance. The query engine automatically rewrites queries to use materialized views when appropriate.

Agent metadata in metric views

Add synonyms, display names, and formatting rules to improve AI agent accuracy and dashboard readability.

Manage and reference

Use the following pages to manage metric view access and find syntax reference information.

Manage metric views

Ask Genie

Control access, enable collaborative editing, and manage the metric view lifecycle using Catalog Explorer or SQL.

Metric view YAML syntax reference

Complete reference for the metric view YAML specification, including all fields, types, and formatting examples.

this page

as this page helpful?

Yes | No

Send feedback

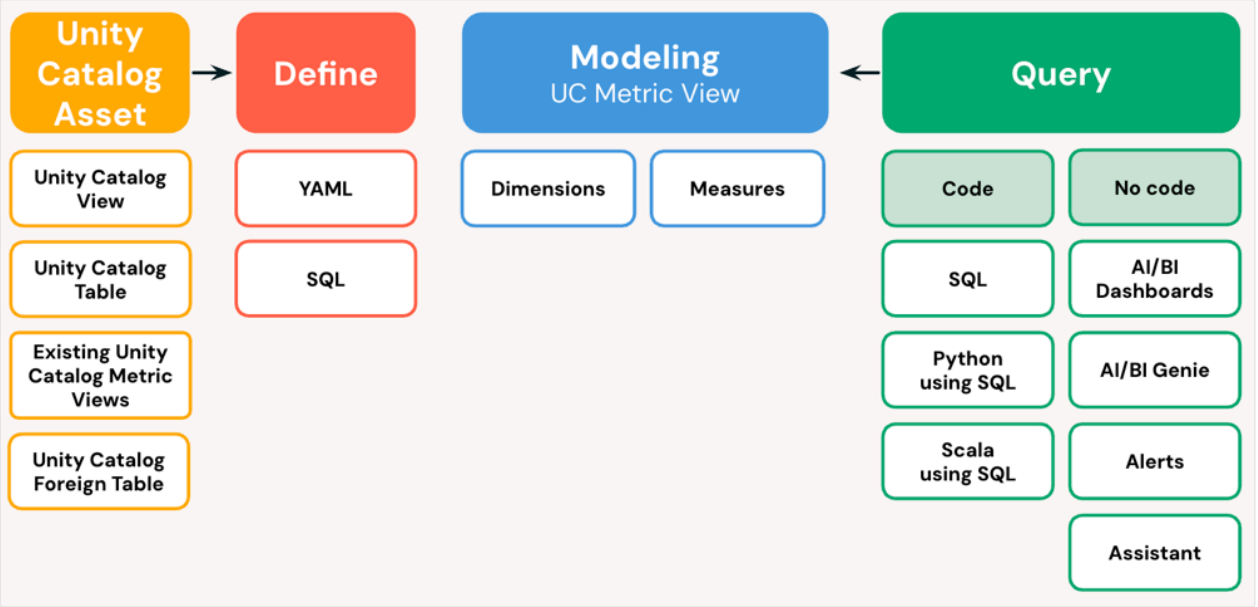
Ask Genie

Last updated on ***Apr 27, 2026***

Unity Catalog metric views

Metric views are the core implementation of Unity Catalog business semantics in Unity Catalog. They provide a centralized way to define and manage business metrics by separating measure definitions from dimension groupings, so you can define metrics once and query them across any dimension at runtime.

Standard views lock in aggregations and dimensions at creation time. With a metric view, you define the metric once — for example, *sum of revenue divided by distinct customer count* — and users can group by any available dimension. The query engine generates the correct computation.



Get started

Use the following pages to create, query, and manage metric views.

Create and edit metric views

Define metric views using SQL DDL or the Catalog Explorer UI, with built-in YAML valid

Ask Genie

Query metric views

Query metric views from SQL editors, notebooks, dashboards, Genie Spaces, and alerts.

Manage metric views

Control access, enable collaborative editing, and manage the metric view lifecycle.

Tutorial: Build a complete metric view with joins

Walk through a complete sales analytics metric view with joins, measures, and agent metadata.

Core features

Use the following pages to learn about modeling, materialization, and agent metadata.

Model metric views

Define sources, dimensions, measures, and filters. Model relationships using star and snowflake schemas with multi-level joins.

Advanced techniques for metric views

Build complex metrics using composability and window measures for trailing averages, period-over-period changes, and cumulative totals.

Materialization for metric views

Pre-compute and incrementally refresh aggregations. The query engine automatically rewrites queries to use materialized views when appropriate. Experimental.

Agent metadata in metric views

Add synonyms, display names, and formatting rules to improve AI agent accuracy and ensure consistent metric display across tools.

Metric view YAML syntax reference

Complete reference for the metric view YAML specification, including all fields, types, and formatting examples.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 7, 2026***

Agent metadata in metric views

Agent metadata (also known as semantic metadata) enhances data visualization and improves large language model (LLM) accuracy by providing display names, format specifications, and synonyms that give business context to your metrics. This metadata helps visualization tools and natural language tools like Genie Spaces interpret and work with your data more effectively.

> **NOTE**
>
> Requires Databricks Runtime 17.3 and YAML version 1.1. See version requirements.

What is agent metadata?

Agent metadata includes display names, format specifications, and synonyms that provide additional context. This metadata helps visualization tools, such as AI/BI dashboards, and natural language tools, such as Genie Spaces, interpret and work with your data more effectively. Agent metadata is defined in the metric view YAML definition.

> **NOTE**
>
> When you create or alter metric views with specification version 1.1, any single-line comments (denoted with `#`) in the YAML definition are removed when the definition is saved. See Upgrade to YAML 1.1 for options and recommendations when upgrading existing YAML definitions.

The examples on this page use the TPC-H sample dataset (`samples.tpch.orders`), which is available by default in Unity Catalog datasets. The TPC-H datase
wholesale supply chain with tables for orders, customers, suppliers, and

names in the `orders` table use the `o_` prefix (for example, `o_orderdate` for order date, `o_totalprice` for total price). For details about the TPC-H schema and data model, see Tutorial: Build a complete metric view with joins.

Display names

Display names provide human-readable labels that appear in visualization tools instead of technical column names. Display names are limited to 255 characters.

The following example shows display names defined on the `order_date` dimension (tracking when orders were placed) and `total_revenue` measure (calculating the sum of all order prices).

YAML

```yaml
version: 1.1
source: samples.tpch.orders

dimensions:
  - name: order_date
    expr: o_orderdate
    display_name: 'Order Date'

measures:
  - name: total_revenue
    expr: SUM(o_totalprice)
    display_name: 'Total Revenue'
```

Synonyms

Synonyms help LLM tools, such as Genie, discover dimensions and measures through user input by providing alternative names. You can define synonyms using either block style or flow style YAML. Each dimension or measure can have up to 10 synonyms. Each synonym is limited to 255 characters.

The following example shows synonyms defined on the `order_date` dimension (when orders were placed) and `total_revenue` measure (sum of all order prices). The

synonyms allow users to ask questions using natural language such as "show me revenue by order time" or "what are total sales by date of order":

YAML

```yaml
version: 1.1
source: samples.tpch.orders

dimensions:
  - name: order_date
    expr: o_orderdate
    # block style
    synonyms:
      - 'order time'
      - 'date of order'

measures:
  - name: total_revenue
    expr: SUM(o_totalprice)
    # flow style
    synonyms: ['revenue', 'total sales']
```

Format specifications

Format specifications define how values should be displayed in visualization tools. The following tables include supported format types and examples.

Numeric formats

Format Type	Required Options	Optional Options
Number: Use plain number format for general numeric values with optional decimal place control and abbreviation options.	`type: number`	• `decimal_places`: Controls the number of places shown after the decimal. ◦ `type`: (Required if `decimal_places` is specified) ▪ `max` ▪ `exact`

Format Type	Required Options	Optional Options
		▪ `all` ◦ `places`: Integer value from 0-10 (required if type is `max` or `exact`) • `hide_group_separator`: When set to true, removes any applicable number grouping separator, such as a `,`. ◦ `true` ◦ `false` • `abbreviation`: ◦ `none` ◦ `compact` ◦ `scientific`
Currency: Use currency format for monetary values with ISO-4217 currency codes.	`type: currency`	• `currency_code`: ISO-4217 code (required). For example, the following codes insert the symbol for US dollars, Euros, and Yen, respectively. ◦ `USD` ◦ `EUR` ◦ `JPY` • `decimal_places`: Controls the number of places shown after the decimal. ◦ `type`: (Required if `decimal_places` is specified) ▪ `max` ▪ `exact` ▪ `all` • `hide_group_separator`: When set to true, removes any applicable

Format Type	Required Options	Optional Options
		number grouping separator. ◦ `true` ◦ `false` • `abbreviation`: ◦ `none` ◦ `compact` ◦ `scientific`
Percentage: Use percentage format for ratio values expressed as percentages.	`type: percentage`	• `decimal_places`: Controls the number of places shown after the decimal. ◦ `type`: (Required if `decimal_places` is specified) ▪ `max` ▪ `exact` ▪ `all` • `hide_group_separator`: When set to true, removes any applicable number grouping separator. ◦ `true` ◦ `false`
Byte: Use byte format for data size values displayed with appropriate byte units (KB, MB, GB, etc.).	`type: byte`	• `decimal_places`: Controls the number of places shown after the decimal. ◦ `type`: (Required if `decimal_places` is specified) ▪ `max` ▪ `exact` ▪ `all` ◦ `places`: Integer value from 0-10 (required if type is `max` or `exact`)

Format Type	Required Options	Optional Options
		• `hide_group_separator`: When set to true, removes any applicable number grouping separator. ◦ `true` ◦ `false`

Numeric formatting examples

Number | Currency | Percentage | Byte

YAML

```yaml
format:
  type: number
  decimal_places:
    type: max
    places: 2
  hide_group_separator: false
  abbreviation: compact
```

Date and time formats

The following table explains how to work with date and time formats.

Format Type	Required Options	Optional Options

Date: Use date format for date values with various display options.	• `type: date`
	• `date_format`: Controls the way the date is displayed
	  ◦ `locale_short_month`: Displays the date with an abbreviated month
	  ◦ `locale_long_month`: Displays the date with the full name of the month
	  ◦ `year_month_day`: Formats the date as YYYY-MM-DD
	  ◦ `locale_number_month`: Displays the date with a month as a number
	  ◦ `year_week`: Formats the date as a year and a week number. For example, `2025-W1`
		• `leading_zeros`: Controls whether single digit numbers are preceded by a zero
		• `true`
		• `false`

DateTime: Use datetime format for timestamp values combining date and time.	• `type: date_time`
	• `date_format`: Controls the way the date is displayed
	  ◦ `no_date`: Date is hidden
	  ◦ `locale_short_month`: Displays the date with an abbreviated month
	  ◦ `locale_long_month`: Displays the date with the full name of the month
	  ◦ `year_month_day`: Formats the date as YYYY-MM-DD
	  ◦ `locale_number_month`: Displays the date with a month as a number
		• `leading_zeros`: Controls whether single digit numbers are preceded by a zero
		  ◦ `true`
		  ◦ `false`

Format Type	Required Options	Optional Options
	◦ `year_week`: Formats the date as a year and a week number. For example, `2025-W1` • `time_format`: ◦ `no_time`: Time is hidden ◦ `locale_hour_minute`: Displays the hour and minute ◦ `locale_hour_minute_second`: Displays the hour, minute, and second	

> **NOTE**
>
> When working with a `date_time` type, at least one of `date_format` or `time_format` must specify a value other than `no_date` or `no_time`.

Datetime formatting examples

Date | DateTime

YAML

```yaml
format:
  type: date
  date_format: year_month_day
  leading_zeros: true
```

Downstream tools integration

Semantic metadata automatically populates downstream tools that consume the metric view:

- **AI/BI dashboards**: Display names and format specifications are automatically populated in dashboard datasets and visualizations to improve dashboard readability.
- **Genie Spaces**: Synonyms are automatically imported to help Genie better discover and understand available dimensions and measures from the metric view.

Complete example

The following example shows a metric view definition that tracks sales performance and includes all agent metadata types. The metric view analyzes order data to calculate revenue metrics, segment customers by order value, and track order volumes.

Customer segments are defined as follows:

- Enterprise: Orders over $100,000
- Mid-market: Orders between $10,000 and $100,000
- Small and medium-sized businesses: Orders under $10,000

The metadata supports natural language queries such as "show me total sales by customer segment" or "what's the average revenue per order."

YAML

```yaml
version: 1.1
source: samples.tpch.orders
comment: Comprehensive sales metrics with enhanced semantic metadata
dimensions:
  - name: order_date
    expr: o_orderdate
    comment: Date when the order was placed
    display_name: Order Date
    format:
      type: date
      date_format: year_month_day
      leading_zeros: true
    synonyms:
      - order time
      - date of order
```

```yaml
  - name: customer_segment
    expr: |
      CASE
        WHEN o_totalprice > 100000 THEN 'Enterprise'
        WHEN o_totalprice > 10000 THEN 'Mid-market'
        ELSE 'SMB'
      END
    comment: Customer classification based on order value
    display_name: Customer Segment
    synonyms:
      - segment
      - customer tier
measures:
  - name: total_revenue
    expr: SUM(o_totalprice)
    comment: Total revenue from all orders
    display_name: Total Revenue
    format:
      type: currency
      currency_code: USD
      decimal_places:
        type: exact
        places: 2
      hide_group_separator: false
      abbreviation: compact
    synonyms:
      - revenue
      - total sales
      - sales amount
  - name: order_count
    expr: COUNT(1)
    comment: Total number of orders
    display_name: Order Count
    format:
      type: number
      decimal_places:
        type: all
      hide_group_separator: true
    synonyms:
      - count
      - number of orders
  - name: avg_order_value
    expr: SUM(o_totalprice) / COUNT(1)
    comment: Average revenue per order
    display_name: Average Order Value
    format:
      type: currency
      currency_code: USD
      decimal_places:
        type: exact
```

```
      places: 2
  synonyms:
    - aov
    - average revenue
```

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie